arXiv:2001.01102v2 [cs.LG] 9 Jan 2020

MushroomRL: Simplifying Reinforcement Learning Research

Carlo D'Eramo[1], Davide Tateo[1], Andrea Bonarini[2], Marcello Restelli[2], Jan Peters[1,3]

[1]IAS Laboratory, TU Darmstadt, Darmstadt, Germany

[2]DEIB, Politecnico di Milano, Milano, Italy

[3]Max Planck Institute for Intelligent Systems, Tübingen, Germany

January 2020

Abstract

MushroomRL is an open-source Python library developed to simplify the process of implementing and running Reinforcement Learning (RL) experiments. Compared to other available libraries, MushroomRL has been created with the purpose of providing a comprehensive and flexible framework to minimize the effort in implementing and testing novel RL methodologies. Indeed, the architecture of MushroomRL is built in such a way that every component of an RL problem is already provided, and most of the time users can only focus on the implementation of their own algorithms and experiments. The result is a library from which RL researchers can significantly benefit in the critical phase of the empirical analysis of their works. MushroomRL stable code, tutorials and documentation can be found at `https://github.com/MushroomRL/mushroom-rl`.

1 Introduction

The advantages of Reinforcement Learning (RL) [9] methodologies are mostly shown in terms of empirical performance, especially in the recent years after the emergence of Deep RL [7]. Indeed, in the vast majority of research papers, experimental evaluation really makes a difference between a successful and well cited work, and a mostly unknown one. The need of evaluating algorithms comes together with the necessity of implementing and testing them in a quick and reliable way; thus, to address these problems, several RL libraries have been developed and are currently used by researchers. However, these libraries have heterogeneous drawbacks, e.g. they only focus on benchmarking and are not easy to extend; they do not have a sufficiently large number of already implemented algorithms.

We introduce MushroomRL, a RL Python library that has been developed to create a flexible, easy to understand, and comprehensive RL framework.

MushroomRL comes with a strongly modular architecture that makes it easy to understand how each component is structured and how it interacts with other ones; moreover it provides an exhaustive list of RL and Deep RL methodologies that are ready to be used as baselines. In this paper, we provide an overview of the most famous RL libraries and briefly compare them with MushroomRL; then, we present the main ideas of our library and how they make MushroomRL powerful and unique; finally, we describe a complex use case to provide an overview of the way MushroomRL can be helpful in implementing and testing a novel complex RL algorithm.

2 Related works

The number of open-source RL libraries has significantly increased consequently to the success of Deep RL. *OpenAI Baselines* [3] is one of the most famous examples of Deep RL libraries. It implements the majority of the most recent techniques and allows to test them on well-known RL problems through an interface with the benchmarking framework OpenAI Gym [2]. The main recognized drawbacks of this library are its complex architecture and the difficulty in understanding the code, thus researchers are discouraged to extend it with novel functionalities and just use it for running available baselines. The more recent *Stable Baselines* [5] is a fork of OpenAI Baselines specifically made with the purpose of simplifying its architecture. However, despite the simplified interface, extending it with novel methodologies is still not straightforward. Another library is *KerasRL* [8] which is built on the well-known Deep Learning library *Keras*. Unfortunately, this library is not maintained anymore. *ChainerRL* is a Deep RL library based on the Deep Learning library *Chainer*. Considering its structure and ideas, ChainerRL can be compared to Keras RL, but is still well maintained and documented. An example of a flexible RL library is *Tensorforce* [6], which is strongly based on Tensorflow. This library implements several Deep RL algorithms and its structure allows to easily test them on custom problems, besides the already available ones. Moreover, its modular architecture facilitate the process of extending the library with novel methodologies. Unfortunately, this library lacks of a complete documentation. Eventually, older RL libraries and interfaces have been proposed in the past, such as: *RL-Glue* [10], *RLPy*, *RLLab* [4]. However, these are not supporting most recent Deep RL techniques and moreover most of them are abandoned projects.

3 Ideas and content

MushroomRL is developed with the explicit goal of addressing all the issues of similar RL libraries. The following are the main qualities of MushroomRL that make it a unique, yet powerful way of performing RL empirical research.

	Value-based	Policy-search
RL	Q-Learning, Double Q-Learning, Weighted Q-Learning, Speedy Q-Learning, R-Learning, SARSA, Expected SARSA, True Online SARSA-λ, FQI, LSPI	REINFORCE, GPOMDP, eNAC, RWR, PGPE, REPS, COPDAC-Q, Stochastic Actor-Critic
Deep RL	DQN, Double-DQN, Averaged-DQN, Categorical DQN	A2C, DDPG, TD3, SAC, TRPO, PPO

Table 1: Some of the available algorithms.

General purpose The research on RL is not only about Deep RL. Shallow RL techniques (e.g. Q-Learning) are still important algorithms to consider, but most of RL libraries ignore them. Since there are no fundamental differences between shallow RL and Deep RL, MushroomRL adapts to heterogeneous learning tasks just focusing in modeling the interaction of an *agent* with an *environment*. This is achieved by a common interface which unifies a broad variety of RL techniques, such as: batch and online algorithms, episodic and infinite horizon tasks, on-policy and off-policy learning, shallow and Deep RL (see Table 1).

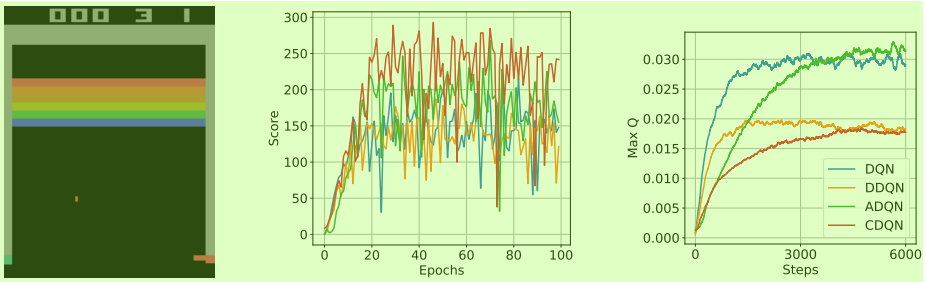
Lightweight MushroomRL is both user-friendly and flexible: only a high-level interface is exposed to the user, hiding low-level aspects. For instance, the user should not care about the implementation details to use a function regressor for different tasks, since they are hidden by a simple common interface. However, we leave the check of consistency constraints to the user, e.g. avoiding the use of a tabular algorithm for an environment with continuous state space. Minimal interfaces simplify the implementation of new algorithms, as there are no hard constraints in the prototypes.

Compatible Standard Python libraries useful for RL tasks have been adopted:

- *Scientific calculus*: Numpy, Scipy;
- *Basic ML*: Scikit-Learn;
- *RL benchmark*: OpenAI Gym, DeepMind Control Suite [11], Pybullet, MuJoCo [12], ROS;
- *Neural networks and GPU computation*: PyTorch.

MushroomRL provides an interface to these libraries, in order to integrate their functionalities in the framework, e.g. an interface for gym environments, support for regression with scikit-learn models.

Easy to use MushroomRL enables to develop and run experiments writing a minimal amount of code. In most of the tasks an experiment can be written in a few Python lines without the need of complex configuration files. The



(a) Rendering of the game (b) Scores (c) Mean maximum Q-function

Figure 1: Rendering and some results on Breakout of DQN and some of its variants.

majority of the RL problems can be solved with experiments written following the structure of the library examples.

4 Advanced use case: solving complex Reinforcement Learning problems

The `examples` folder in MushroomRL provides several already implemented scripts that the user can run, and use as a reference to write their own. To show how easily a user can write and run complex experiments in MushroomRL, we consider the `atari_dqn.py` script, which is used to run Deep Q-Network (DQN) algorithm [7] on the Atari benchmark [1]. This script generally follows the structure we recommend

```
# Instantiation of problem, policy, and function approximator
mdp = Atari()
pi = EpsGreedy(...)
nn = TorchApproximator
# Instantiation of the agent (i.e. algorithm)
agent = DQN(approximator=nn, policy=pi, ...)
# Instantiation of the core
core = Core(agent, mdp)
# Learning and evaluation
core.learn(...)
core.evaluate(...)
```

with some addition, e.g. the alternation between learn and evaluation phases as described in [7], and provide some useful functionalities that are easily integrated in the script, e.g. the use of the `argparse` library to parse input from the command line. Moreover, it saves the weights of the trained network, which can be loaded to initialize another network and evaluate it. Furthermore,

MushroomRL leaves the possibility to rely on other useful functionalities like plotting (e.g. using `matplotlib` library), or running multiple experiments in parallel (e.g. using `joblib` library). Figure 1 shows some results obtained running `atari_dqn.py` with DQN and four of its variants; note the high variance in the scores and the difference between the Q-values of DQN and DDQN, as described in literature. These results can be easily collected in MushroomRL through the use of the already provided callback functions, or new ones that the user can add. As stated in Section 3, generally in MushroomRL only the low-level modules should not be modified by the user, while all the other ones can be customized and most of the functionalities provided by external libraries can be used, giving the user much freedom about the structure and analysis of his/her experiments. In the case the user wants to add his/her own algorithm, he/she can simply add it in the `mushroom_rl/algorithms/` and instantiate it in the script of the experiment.

5 Conclusion

We presented MushroomRL, an RL library to help researchers to easily develop their works and compare the results w.r.t. the majority of classical RL techniques and Deep RL approaches. Full documentation, installation instructions, and tutorials are available at `http://mushroomrl.readthedocs.io/en/latest`.

References [1]

[1] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. 1 6

[2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016. 8

[3] Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu, and Peter Zhokhov. Openai baselines. `https://github.com/openai/baselines`, 2017. 10

[4] Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and Pieter Abbeel. Benchmarking deep reinforcement learning for continuous control. In *International Conference on Machine Learning*, pages 1329–1338, 2016. 11

[5] Ashley Hill, Antonin Raffin, Maximilian Ernestus, Adam Gleave, Rene Traore, Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, and Yuhuai Wu. Stable baselines. `https://github.com/hill-a/stable-baselines`, 2018. 2

[6] Alexander Kuhnle, Michael Schaarschmidt, and Kai Fricke. Tensorforce: a tensorflow library for applied reinforcement learning. Web page, 2017. URL `https://github.com/tensorforce/tensorforce`. 5

[7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. 4

[8] Matthias Plappert. keras-rl. `https://github.com/keras-rl/keras-rl`, 2016. 3

[9] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998. 13

[10] Brian Tanner and Adam White. Rl-glue: Language-independent software for reinforcement-learning experiments. *Journal of Machine Learning Research*, 10(Sep):2133–2136, 2009. 7

[11] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018. 9

[12] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. 12